

Parallel and High Performance Computing

Dr. Pablo Antolín

Final Project

April 30 2026

Lattice Boltzmann method: 2D flow past a cylinder

1 Problem description

The Lattice Boltzmann Method (LBM) is a kinetic-theory based scheme that solves fluid flow problems by tracking discrete particle distributions on a regular grid (https://en.wikipedia.org/wiki/Lattice_Boltzmann_methods). In this project you will work with the standard **D2Q9** lattice: a two-dimensional grid where each cell carries nine particle distributions $f_i(\mathbf{x}, t)$, $i = 0, \dots, 8$, one per discrete velocity $\mathbf{c}_i$ (Figure 1). Using the index convention of that figure, the velocities are

$$(\mathbf{c}_0, \dots, \mathbf{c}_8) = (0, 0), (1, 0), (0, 1), (-1, 0), (0, -1), (1, 1), (-1, 1), (-1, -1), (1, -1).$$

The macroscopic density $\rho(\mathbf{x}, t)$ and the macroscopic fluid velocity $\mathbf{u}(\mathbf{x}, t)$ are recovered from the distributions as

$$\rho(\mathbf{x}, t) = \sum_{i=0}^8 f_i(\mathbf{x}, t), \quad \rho(\mathbf{x}, t) \mathbf{u}(\mathbf{x}, t) = \sum_{i=0}^8 \mathbf{c}_i f_i(\mathbf{x}, t).$$

Each time step alternates two operations:

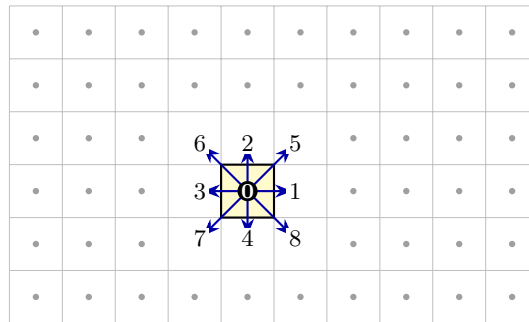


Figure 1: D2Q9 lattice (a 10×6 excerpt; in practice the full grid is $n_x \times n_y$ cells). Every cell carries nine particle distributions f_i , $i = 0, \dots, 8$, one per discrete velocity $\mathbf{c}_i$. The numbering on the highlighted cell matches the convention used in the provided code (`lbm.cc`): $i = 0$ is the rest direction, $i = 1 \dots 4$ are the four axis-aligned directions, and $i = 5 \dots 8$ are the four diagonals. The streaming step shifts each f_i from its cell to the neighbour along $\mathbf{c}_i$.

1. *Collision* — relax every cell towards the local equilibrium with a single relaxation time τ (BGK):

$$f_i^*(\mathbf{x}, t) = f_i(\mathbf{x}, t) - \frac{1}{\tau} [f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\rho, \mathbf{u})].$$

For the D2Q9 lattice the equilibrium has the closed form

$$f_i^{\text{eq}}(\rho, \mathbf{u}) = w_i \rho \left[1 + 3 (\mathbf{c}_i \cdot \mathbf{u}) + \frac{9}{2} (\mathbf{c}_i \cdot \mathbf{u})^2 - \frac{3}{2} |\mathbf{u}|^2 \right],$$

with weights $w_0 = 4/9$, $w_{1\dots 4} = 1/9$, $w_{5\dots 8} = 1/36$ for the rest, axis, and diagonal directions respectively.

2. *Streaming* — push the post-collision distributions to the neighbouring cells along their velocity:

$$f_i(\mathbf{x} + \mathbf{c}_i, t + 1) = f_i^*(\mathbf{x}, t).$$

Combining both steps yields the single lattice Boltzmann update

$$f_i(\mathbf{x} + \mathbf{c}_i, t + 1) = f_i(\mathbf{x}, t) - \frac{1}{\tau} [f_i(\mathbf{x}, t) - f_i^{\text{eq}}(\rho, \mathbf{u})],$$

which can be read as advection of each population along its velocity with relaxation towards local equilibrium. The relaxation time controls the kinematic viscosity $\nu = c_s^2(\tau - \frac{1}{2})$, where $c_s = 1/\sqrt{3}$ is the *lattice speed of sound* and $p = \rho c_s^2 = \rho/3$ the lattice pressure. The Reynolds number is $\text{Re} = u_{\text{in}} D / \nu$. Two practical constraints follow: stability requires $\tau > \frac{1}{2}$, and the incompressible approximation holds only when the Mach number is small, $\text{Ma} = u_{\text{in}} / c_s = u_{\text{in}} \sqrt{3} \ll 1$ (with $u_{\text{in}} = 0.05$ one has $\text{Ma} \approx 0.09$). A Chapman–Enskog multiscale expansion shows that LBM recovers the incompressible Navier–Stokes equations at second order in the Mach number.

No-slip boundary condition. At solid cells the post-collision distributions are reflected back before streaming (*halfway bounce-back*):

$$f_{\bar{i}}(\mathbf{x}, t) \leftarrow f_i^*(\mathbf{x}, t), \quad s(\mathbf{x}) = 1,$$

where $\bar{i}$ is the index diametrically opposite to i (e.g. $\bar{1} = 3$, $\bar{5} = 7$). Reversing the particle momentum at the surface enforces zero velocity (no-slip) there. From the perspective of parallelisation, the key property of the whole algorithm is that every update is *purely local*: each cell reads only itself and its eight immediate neighbours, making the scheme a stencil sweep on a regular grid.

2 Basic sequential algorithm

The full time integration of the solver is summarised below.

Data: lattice $f_i(\mathbf{x}, 0)$, solid mask $s(\mathbf{x}) \in \{0, 1\}$ (1 in the no-slip walls and the obstacles, 0 in the fluid), τ , u_{in} , number of steps N

Result: lattice $f_i(\mathbf{x}, N)$ and macroscopic fields $\rho, \mathbf{u}$

initialise $f_i(\mathbf{x}, 0) \leftarrow f_i^{\text{eq}}(\rho=1, \mathbf{u}=(u_{\text{in}}, 0))$ on every cell with $s(\mathbf{x}) = 0$;

```

for  $n = 1, \dots, N$  do
    // collide
    foreach cell  $\mathbf{x}$  do
        compute  $\rho(\mathbf{x})$  and  $\mathbf{u}(\mathbf{x})$  from  $f_i(\mathbf{x})$ ;
         $f_i(\mathbf{x}) \leftarrow f_i(\mathbf{x}) - \frac{1}{\tau}[f_i(\mathbf{x}) - f_i^{\text{eq}}(\rho, \mathbf{u})]$ ;
    end
    // bounce-back inside obstacles and on no-slip walls
    foreach cell  $\mathbf{x}$  with  $s(\mathbf{x}) = 1$  do
        swap each pair of opposite directions  $f_i \leftrightarrow f_{\bar{i}}$ ;
    end
    // stream
    foreach cell  $\mathbf{x}$  and direction  $i$  do
         $f_i(\mathbf{x}, n) \leftarrow f_i(\mathbf{x} - \mathbf{c}_i, n - 1)$ ;
    end
    // boundary conditions
    inlet:  $f_i \leftarrow f_i^{\text{eq}}(\rho=1, \mathbf{u}=(u_{\text{in}}, 0))$  on the left edge;
    outlet: copy the second-to-last column into the last on the right edge;
end

```

The flow problem. The simulation domain is a rectangular channel of $n_x \times n_y$ cells (Figure 2). The top and bottom walls are no-slip; the inlet (left edge) prescribes a uniform horizontal velocity u_{in} ; the outlet (right edge) is a zero-gradient copy of its left neighbour. One (or two) circular *cylinders* are placed inside the channel as the obstacles, where populations are reflected by simple bounce-back. At $\text{Re} \approx 100$ the wake behind the cylinder becomes unstable and develops a periodic vortex pattern, the *Karman vortex street* (https://en.wikipedia.org/wiki/Karman_vortex_street; Figure 3). The non-dimensional shedding frequency,

$$\text{St} = \frac{f_{\text{shed}} D}{u_{\text{in}}},$$

is called the Strouhal number; for an isolated cylinder at $\text{Re} = 100$ the literature reports $\text{St} \approx 0.16$. You will use this value as the validation target.

3 The provided code

A serial C++11 implementation of the solver is provided in the course git repository. It contains:

- `lbm.hh`, `lbm.cc` — the D2Q9 BGK solver. Distributions are stored as a *structure of arrays*: nine contiguous slabs of $n_x \cdot n_y$ doubles. The sweep is split into `collide()`, `bounce_back()`, `stream()`, `apply_inlet()`, `apply_outlet()`.
- `output.hh`, `output.cc` — writes one HDF5 file per snapshot and a single XDMF index file (`.xdmf`) that ParaView (and PyVista) read directly.

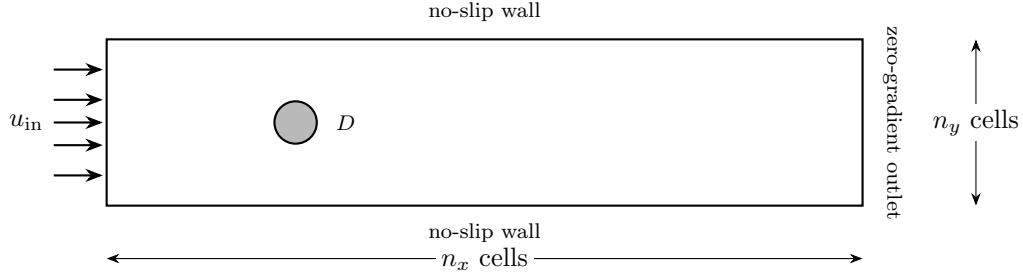


Figure 2: Channel-with-cylinder geometry (not to scale). The inlet on the left prescribes a uniform velocity u_{in} ; the outlet on the right is a zero-gradient copy of its left neighbour. The cylinder diameter D is the reference length used in the Reynolds and Strouhal numbers.

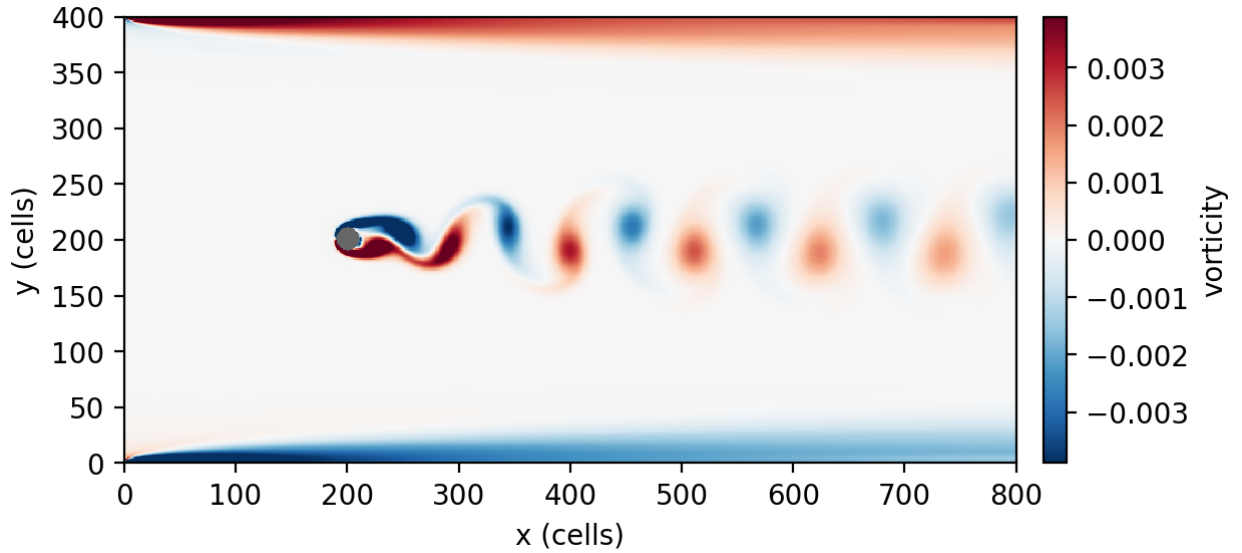


Figure 3: Vorticity field from the serial reference at the default geometry ($n_x = 800$, $n_y = 400$, $\text{Re} = 100$, $u_{\text{in}} = 0.05$) after 60 000 time steps. Red and blue indicate counter-rotating vortices being shed alternately from each side of the cylinder; the resulting wake is the Karman vortex street.

- `main.cc` — a small command-line driver. All parameters (n_x , n_y , Re , u_{in} , the cylinder location and radius, the number of steps, the output cadence, the probe location...) are passed as `key=value` pairs.
- `Makefile` — a plain Makefile, depends only on a working HDF5 installation.
- `viz/viz.py` — a Python script that consumes the XDMF file with PyVista and renders an animated GIF of the chosen field (vorticity by default).
- `viz/strouhal.py` — a Python script that reads the probe CSV, runs an FFT, and prints the measured Strouhal number. Use this to validate your implementation.

A short `README.md` in the same folder explains how to build and run the code. Read both that and the source files carefully *before* you start parallelizing anything.

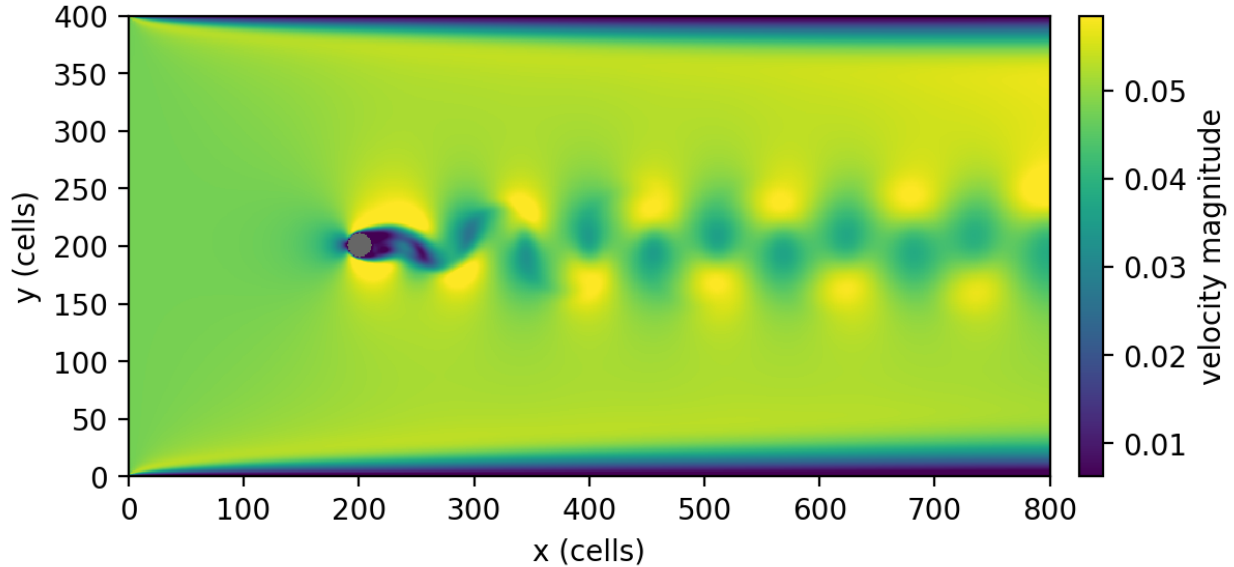


Figure 4: Velocity magnitude $|\mathbf{u}|$ at the same instant as Figure 3. The alternating bright and dark patches in the wake reveal the high- and low-speed lobes that surround each shed vortex; far from the cylinder the flow recovers the imposed inlet velocity $u_{\text{in}} = 0.05$.

4 The project

You have to develop **two parallel implementations** of this code:

1. A distributed-memory version using **MPI**.
2. An accelerated version using **CUDA** with hand-written kernels.

There is no need to parallelize Python post-processing files.

Before writing any parallel code, profile the serial version (e.g. with **gprof**, **perf**, or **Intel VTune**) and identify which parts dominate the run time. The structure of the code is intentionally simple: do not refactor it speculatively — let the profile guide which loops are worth parallelizing first. Use Amdahl’s and Gustafson’s laws to predict the achievable scaling before you measure it.

Choice of problem sizes. The serial code accepts n_x and n_y as command-line arguments. Pick sizes that produce a meaningful study: small enough that you can run many configurations, large enough to amortise communication and synchronisation costs in the parallel versions. The default geometry is sized for a quick smoke test, not for a scaling study.

Validation. For both your parallel versions you must reproduce the canonical Karman vortex shedding at $\text{Re} = 100$ and recover $\text{St} \approx 0.16$ using the provided **strouhal.py** script. The default geometry in **main.cc** already gives this value within a few percent in serial.

5 Performance analysis

The performance study — not just the implementation — is the heart of this project. Your report must include at least the following:

- The standard LBM throughput metric, *million lattice updates per second* (MLUPS), is printed by the serial code at the end of every run. Report it for the serial baseline and for each parallel configuration.
- A **strong scaling** study of the MPI version, against your own Amdahl prediction, for several representative problem sizes.
- A **weak scaling** study of the MPI version, against Gustafson’s law.
- A study of the CUDA version: how performance depends on the block / grid configuration, on the problem size, and on the chosen memory layout. Report the achieved MLUPS and discuss it against an order-of-magnitude bound for the V100 GPU you will use (e.g. a rough roofline estimate from peak memory bandwidth).

If at any point your numbers contradict the theoretical predictions (super-linear speed-up, MLUPS above the memory-bandwidth bound, weak scaling efficiency above 100%...), *say so explicitly* and investigate. Anomalies are interesting; pretending they do not exist is not.

6 Going further

The requirements above, met correctly and thoroughly, already constitute a strong project. The last fraction of the grade is reserved for work that goes beyond them in a direction you choose yourself. There is no prescribed path. What matters is that the extension is well motivated, executed carefully, and clearly argued in the report. A small but well-motivated study, executed carefully, is worth more than a long list of unfinished ideas.

7 Submission

You submit the project on Moodle as three different files (one PDF for the report, another PDF for the slides, and a tarball for the code):

- The report (PDF): body text at most **1500 words**, with **at most 5 figures and tables** combined (references are excluded from the word count). It has to include a short *AI usage statement* (excluded from word count). List which AI tools you used and for which parts of the work, and explain how you checked their output. Honest and specific disclosures are not penalized.
- The source code of your two parallel versions, in two clearly separated folders. It should contain a short `README.md` with build and run instructions for each version.
- Your slides (PDF), at most 4 slides:
 1. Strong scaling of the MPI version.
 2. Weak scaling of the MPI version.
 3. CUDA performance.
 4. One slide of your choice (e.g. a result, a plot, an observation you want to discuss).

Oral exam. You will give a short presentation (about 5 minutes) of the four slides listed above. The presentation is followed by 5–10 minutes of questions about the project. General questions about the course may be asked too. Be prepared to explain why you made the implementation choices you made, and discuss specific lines of your code.

Deadlines.

- Code, report and slides: submitted on Moodle before **June 7, 23h59 CEST**.
- Oral exams: scheduled according to EPFL's exam calendar; the precise schedule will be published on Moodle.

8 Computers and support

You will run your code on the SCITAS clusters. Use the `--account=math-454` flag for your Slurm submissions and the reservation announced on Moodle.

Office hours will be on April 30th, May 7th, May 21st, and May 28th, from 1:15 to 5:00 PM in GR A3 30. Beyond that period, replies to questions are not guaranteed.